Dual Model Pipeline — 코드 & 기능 설명

> 대상 파일: `pipelines/dual_model_pipeline.py` (단일 파일 1,552 lines) UI(`chat_ui.html`, `server.py`) 는 다루지 않고, 파이프라인 코드의 함수 단위 동작만 정리한 문서.

목차

1. 모듈 상수 & 설정 로더
2. Valves — 런타임 설정
3. 시스템 프롬프트
4. 요청 분류 (코딩 vs 단순 질문)
5. 외부 데이터 수집
6. 대화 히스토리 정규화
7. 일반 채팅 응답 `generate_chat_reply`
8. 자동 라우팅 (복잡도 추정)
9. 메인 파이프 `pipe`
10. Two-stage 모드 `generate_spec` / `generate_code_from_spec`
11. Diff 수정 파이프라인 `pipe_modify`
12. 설정/명세 검증
13. Ollama 호출 (비동기 ↔ 동기 브리지)
14. 명세 저장 & 로드
15. 응답 포맷팅 유틸

---

1. 모듈 상수 & 설정 로더

```
def _now_iso() -> str:
    return datetime.now().astimezone().isoformat(timespec="seconds")

CONFIG_PATH = Path(__file__).resolve().with_name("config.json")
SPECS_ROOT  = Path(__file__).resolve().parent.parent / "specs"


def _load_config() -> dict[str, Any]:
    if not CONFIG_PATH.exists():
        return {}
    try:
        config = json.loads(CONFIG_PATH.read_text(encoding="utf-8"))
    except (json.JSONDecodeError, OSError):
        return {}
    return config if isinstance(config, dict) else {}


CONFIG = _load_config()


def _config_value(*keys: str, default: Any = "") -> Any:
```

```
    for key in keys:
        if key in CONFIG:
            return CONFIG[key]
    return default
```

기능

- `_now_iso()` — 로컬 타임존 기준 ISO 타임스탬프. 명세 저장 레코드의 `saved_at` 필드에 쓰인다.
- `CONFIG_PATH` / `SPECS_ROOT` — 모듈 위치 기준 절대경로. 워킹 디렉터리 바뀌어도 안전.
- `_load_config()` — `pipelines/config.json` 을 읽어서 dict 로 반환. **파일 없음 / 파싱 실패 / dict 가 아님 → 빈 dict 로 폴백**. 설정 파일 하나 때문에 파이프라인이 죽지 않도록 만든 안전장치.
- `CONFIG` — 모듈 import 시점에 한 번만 로드되는 전역 캐시.
- `_config_value(*keys, default)` — 여러 키 후보 중 먼저 존재하는 값을 반환. 예: `_config_value("plan_model", "spec_model")` 는 구/신 키 이름 둘 다 허용.

---

2. Valves — 런타임 설정

```
class Valves(BaseModel):
    model_config = ConfigDict(protected_namespaces=())

    spec_model: str = Field(
        default=str(_config_value("plan_model", "spec_model",
default="llama3.1:8b")),
        description="Spec model name. Ollama: llama3.1:8b",
    )
    coder_model: str = Field(
        default=str(_config_value("dev_model", "coder_model", default="qwen2.5-
coder:7b")),
        description="Coder model name. Ollama: qwen2.5-coder:7b",
    )
    single_pass: bool = Field(
        default=bool(_config_value("single_pass", default=True)),
        description="Use one model call instead of spec+coder calls to reduce API
usage.",
    )
    spec_model_light: str = Field(
        default=str(_config_value("spec_model_light", default="")),
        description="Lightweight spec model for simple requests.",
    )
    coder_model_light: str = Field(
        default=str(_config_value("coder_model_light", default="")),
        description="Lightweight coder model for simple requests.",
    )
    ollama_url: str = Field(
        default=str(_config_value("ollama_url",
default="http://localhost:11434")),
        description="Ollama API base URL.",
    )
    auto_route: bool = Field(
        default=bool(_config_value("auto_route", default=False)),
```

```
        description="Auto-select model size by request complexity",
    )
```

기능

Pipeline 클래스의 내부 Pydantic 모델. Open WebUI 가 **런타임에 값을 바꿀 수 있는 "밸브"** 역할.

필드	의미
`spec_model`	명세 생성용 모델명. `plan_model` / `spec_model` 두 키 이름 모두 인식
`coder_model`	코드 생성용 모델명. `dev_model` / `coder_model` 모두 인식
`single_pass`	`True` 면 Spec + Coder 호출을 합쳐 1회로 끝냄
`spec_model_light` / `coder_model_light`	자동 라우팅 시 "간단한 요청"에 쓰는 경량 모델
`ollama_url`	Ollama 서버 URL
`auto_route`	복잡도 기반 경량/본모델 자동 전환 on/off

`model_config = ConfigDict(protected_namespaces=())` 는 Pydantic v2 의 `model_` prefix 경고를 끄기 위한 설정.

---

3. 시스템 프롬프트

파이프라인이 사용하는 프롬프트는 총 6종이다. 모두 클래스 속성으로 정의되며, 프론트엔드 프레임워크 선택 규칙이 주요 4종의 끝에 concat 된다.

3.1 Few-shot 예시

```
FEW_SHOT_SPEC_EXAMPLE = json.dumps(
    {
        "project": {
            "name": "Todo App",
            "type": "static-web",
            "scope": "frontend",
            "tech_stack": ["HTML", "CSS", "Vanilla JS"],
            "description": "로컬 스토리지 기반 할일 관리 앱",
        },
        "files": [
            {"path": "index.html", "role": "메인 진입점, 전체 레이아웃 구조"},
            {"path": "style.css", "role": "다크모드 지원 스타일시트"},
            {"path": "app.js", "role": "CRUD 로직, 로컬스토리지 연동"},
        ],
        "components": [
            {
                "name": "TodoList",
                "props": ["items: Array", "onDelete: Function"],
```

```
                "behavior": "항목 클릭 시 완료 토글, 완료 항목 취소선 표시",
            }
        ],
        "api": [],
        "constraints": [
            "외부 라이브러리 사용 금지",
            "모바일 반응형 필수",
            "색상 팔레트: #1a1a2e, #16213e, #e94560",
        ],
        "user_story": "사용자는 할일을 추가·완료·삭제할 수 있다",
    },
    ensure_ascii=False,
    indent=2,
)

FEW_SHOT_DIFF_EXAMPLE = json.dumps(
    {
        "modified_files": [
            {
                "path": "style.css",
                "changes": "버튼 색상을 #e94560에서 #00d2ff로 변경",
            }
        ],
        "new_constraints": ["버튼에 hover 애니메이션 추가"],
        "removed_components": [],
    },
    ensure_ascii=False,
    indent=2,
)
```

모듈 로드 시점에 한 번만 JSON 으로 직렬화해 두고, 아래의 `SPEC_SYSTEM_PROMPT` / `DIFF_SPEC_PROMPT` 문자열에 f-string 으로 박아 넣는다. 매 요청마다 재직렬화하지 않도록 최적화.

3.2 프론트엔드 프레임워크 규칙

```
FRONTEND_FRAMEWORK_RULE = """
프론트엔드 프레임워크 선택 규칙 (반드시 아래 3가지 중 하나만 선택):

[방법 1] Vanilla HTML/CSS/JS
- 선택 기준: 단순 정적 페이지, 랜딩 페이지, 학습용 예제, 단일 화면 도구, 작은 유틸리티
- 파일 규칙: **반드시 단일 `.html` 파일 하나**로만 작성합니다.
  * CSS는 `<style>` 태그 안에 인라인으로 작성합니다.
  * JS는 `<script>` 태그 안에 인라인으로 작성합니다.
  * `style.css`, `script.js`, `app.js` 같은 **별도 파일을 절대 만들지 마십시오.**
  * `<link rel="stylesheet" href="...">` 와 `<script src="외부파일.js">` 를 사용하지 마십시오.
  * 외부 CDN 라이브러리(예: CDN 링크)는 허용됩니다.

[방법 2] React
- 선택 기준: 상태 관리가 중요한 SPA, 컴포넌트 재사용이 많은 UI, 동적 라우팅, 복잡한 상호작용, 대시보드
```

```
- 파일 규칙: Vite 기반 프로젝트 구조로 작성합니다.
  * `package.json`, `index.html`, `src/main.jsx`, `src/App.jsx`, 필요 시
`src/components/*.jsx`
  * 스타일은 `src/App.css` 또는 CSS Modules 사용 가능
  * TypeScript가 꼭 필요하지 않으면 `.jsx` 사용

[방법 3] Flutter
- 선택 기준: 모바일 앱, 크로스플랫폼 앱, 네이티브 앱 느낌의 UI, "앱 만들어줘" 요청
- 파일 규칙: Flutter 프로젝트 구조로 작성합니다.
  * `pubspec.yaml`, `lib/main.dart`, 필요 시 `lib/screens/*.dart`,
`lib/widgets/*.dart`

중요:
- 한 응답 안에서 반드시 **하나의 방법만** 선택합니다. 여러 방법을 섞지 마십시오.
- 사용자가 특정 프레임워크를 명시하면 그것을 따릅니다.
- 단순 요청(한 화면짜리, 계산기, 시계, 할일 앱 등)은 기본적으로 방법 1을 선택합니다.
- 백엔드/CLI/데이터 요청에는 이 규칙이 적용되지 않습니다.
""".strip()
```

모델이 "반 Vanilla + 반 React" 같은 혼합 구조를 출력하지 못하도록 강제하는 규칙 문자열. 파일 규칙이 매우 구체적이라(단일 `.html`, Vite 경로 구조, Flutter 디렉터리) LLM 이 의도와 다른 출력을 낼 여지를 줄인다.

3.3 `SPEC_SYSTEM_PROMPT` — 명세 생성용

```
SPEC_SYSTEM_PROMPT = f"""당신은 소프트웨어 명세 작성 전문가입니다.
사용자의 자연어 요청을 분석하여 반드시 아래 JSON 스키마 형식의 기술 명세서만 출력하십시
오.
설명 텍스트, Markdown 코드블록, 주석 없이 순수 JSON만 출력합니다.

필수 스키마:
{{
  "project": {{
    "name": "string",
    "type": "string (vanilla-web | react-spa | flutter-app | node-app | cli | data
중 하나)",
    "scope": "string (frontend | backend | fullstack | cli | data | general 중 하
나)",
    "tech_stack": ["string"],
    "description": "string"
  }},
  "files": [
    {{ "path": "string", "role": "string (이 파일의 역할 설명)" }}
  ],
  "components": [
    {{
      "name": "string",
      "props": ["string"],
      "behavior": "string"
    }}
  ],
  "api": [],
```

```
  "constraints": ["string"],
  "user_story": "string"
}}

scope 규칙:
- 사용자의 요청이 화면/UI/웹페이지 중심이면 "frontend"를 선택합니다.
- API, 서버, DB, 인증, 배치 작업 중심이면 "backend"를 선택합니다.
- 프론트엔드와 백엔드가 모두 필요하면 "fullstack"을 선택합니다.
- 터미널 도구나 스크립트 중심이면 "cli"를 선택합니다.
- 데이터 처리/분석 중심이면 "data"를 선택합니다.
- 어느 쪽에도 명확히 속하지 않으면 "general"을 선택합니다.
- 모든 요청을 프론트엔드로 간주하지 마십시오. scope에 맞는 파일만 설계하십시오.

아래는 올바른 출력 예시입니다:
{FEW_SHOT_SPEC_EXAMPLE}""".strip()
```

자연어 → 고정 스키마의 JSON 명세로 변환. "순수 JSON 만 출력" 을 강하게 지시하고, `scope` 허용 값 6개를 명시해 파이프라인 하류의 `_validate_spec` 와 정합을 맞춘다. Few-shot 예시가 뒤에 붙어 포맷 이탈을 줄인다.

3.4 `CODER_SYSTEM_PROMPT` — 코드 생성용

```
CODER_SYSTEM_PROMPT = """당신은 코드 생성 전문가입니다.
주어진 기술 명세서 JSON을 100% 준수하여 코드를 생성하십시오.
응답에는 먼저 구현 설명을 3~6문장으로 작성하고, 그 다음 실제 파일별 코드를 생성하십시오.
project.scope가 frontend가 아니라면 프론트엔드 파일을 억지로 만들지 말고, 해당 scope에
필요한 서버/CLI/데이터 처리 파일을 생성하십시오.
각 파일은 반드시 아래 형식으로 출력합니다.

```파일경로
코드내용
```
```

명세에 없는 기능은 추가하지 말고, 파일 경로는 명세서의 files 배열에 있는 값만 사용하십시오. 코드는 그대로 저장해 실행할 수 있는 완성본이어야 합니다.""".strip()

```
JSON 명세를 입력받아 파일별 코드 블록으로 출력. ` ```파일경로 ` 형식은 비표준이지만, 후
단의 `_format_code_for_webui` 가 Open WebUI 호환 포맷으로 변환해 준다. "명세에 없는
기능 금지", "파일 경로는 files 배열만 사용" 으로 모델의 임의 확장을 억제.

### 3.5 `SINGLE_PASS_SYSTEM_PROMPT` — 1회 호출 모드

```python
SINGLE_PASS_SYSTEM_PROMPT = """당신은 코드 생성 전문가입니다.
사용자 요청을 내부적으로 분석해 작업 범위를 frontend, backend, fullstack, cli, data,
general 중 하나로 판단하십시오.
판단 결과와 중간 분석 과정은 절대 출력하지 말고, 최종 구현 설명과 파일별 코드만 출력하십
시오.
요청이 프론트엔드가 아니라면 프론트엔드 파일을 억지로 만들지 말고, 해당 범위에 필요한 서
버/CLI/데이터 처리 파일을 생성하십시오.
```
```

```
응답에는 먼저 구현 설명을 3~6문장으로 작성하고, 그 다음 실제 파일별 코드를 생성하십시오.
각 파일은 반드시 아래 형식으로 출력합니다.

```파일경로
코드내용
```
```

코드는 그대로 저장해 실행할 수 있는 완성본이어야 합니다.""".strip()

```
`Valves.single_pass=True` 일 때 사용. SPEC + CODER 를 하나의 호출로 합치기 위해,
scope 판단을 모델 **내부 사고**로 처리하고 최종 코드만 출력하도록 지시. LLM 호출 1회로
끝나 API 비용과 응답 시간을 절반 가까이 줄이지만, 명세 검증이 없어 디버깅 단서는 약해진
다.

### 3.6 `DIFF_SPEC_PROMPT` / `DIFF_CODER_PROMPT` — 수정 요청용

```python
DIFF_SPEC_PROMPT = f"""당신은 소프트웨어 명세서 diff 전문가입니다.
원본 명세서와 사용자의 수정 요청을 비교하여, 변경이 필요한 항목만 담은 diff 명세서를
JSON으로 출력하세요.
설명 텍스트, Markdown 코드블록, 주석 없이 순수 JSON만 출력합니다.

필수 형식:
{{
  "modified_files": [
    {{ "path": "string", "changes": "변경 내용 설명" }}
  ],
  "new_constraints": ["새로 추가할 제약 조건"],
  "removed_components": ["삭제할 컴포넌트 이름"]
}}

아래는 올바른 출력 예시입니다:
{FEW_SHOT_DIFF_EXAMPLE}""".strip()

DIFF_CODER_PROMPT = """당신은 코드 수정 전문가입니다.
기존 코드와 diff 명세서가 주어집니다. diff 명세서에 명시된 변경 사항만 반영하여 수정된
파일을 출력하십시오.
변경되지 않는 파일은 출력하지 마십시오.
각 파일은 반드시 아래 형식으로 출력합니다.

```파일경로
전체코드내용
```
```

diff에 없는 변경은 절대 하지 마십시오.""".strip()

```
- `DIFF_SPEC_PROMPT` — 원본 명세 + 수정 요청을 비교해 **변경이 필요한 항목만** 담은 작
은 JSON 을 뽑는다. 전체 명세 재생성보다 토큰 소비가 훨씬 적다.
```
```

```
- `DIFF_CODER_PROMPT` — 기존 코드 + diff 명세를 받아 **변경된 파일만** 전체 코드로 출력. "변경되지 않는 파일 출력 금지" / "diff 에 없는 변경 금지" 두 가지 제약으로 모델의 범위 초과를 막는다.

### 3.7 `CHAT_SYSTEM_PROMPT` — 일반 채팅용

```python
CHAT_SYSTEM_PROMPT = (
    "당신은 친절하고 간결한 한국어 어시스턴트입니다. "
    "사용자의 질문에 핵심만 명료하게 답하고, 필요한 경우에만 예시를 드세요. "
    "코드가 필요한 경우에만 코드 블록을 사용하고, 불필요하게 길게 쓰지 마십시오. "
    "\"참고 자료\" 블록이 제공되면 그 안의 정보를 우선으로 사용하고, "
    "출처가 있는 경우 답변 끝에 '(출처: ...)' 형태로 표기하세요."
)
```

코딩 요청이 아닌 일반 질문 경로(`generate_chat_reply`) 에서 사용. 외부 데이터 수집 결과(`--- 참고 자료 ---` 블록) 가 들어오는 경우의 동작 규칙까지 함께 정의해 둔다.

3.8 프레임워크 규칙 concat

```python
SPEC_SYSTEM_PROMPT        = f"{SPEC_SYSTEM_PROMPT}\n\n{FRONTEND_FRAMEWORK_RULE}"
CODER_SYSTEM_PROMPT       = f"{CODER_SYSTEM_PROMPT}\n\n{FRONTEND_FRAMEWORK_RULE}"
SINGLE_PASS_SYSTEM_PROMPT = f"
{SINGLE_PASS_SYSTEM_PROMPT}\n\n{FRONTEND_FRAMEWORK_RULE}"
DIFF_CODER_PROMPT         = f"{DIFF_CODER_PROMPT}\n\n{FRONTEND_FRAMEWORK_RULE}"
```

클래스 정의 시점에 한 번만 덧붙여서 최종 프롬프트로 만든다. `DIFF_SPEC_PROMPT` 와 `CHAT_SYSTEM_PROMPT` 는 프론트엔드 관련이 아니므로 제외.

기능

- `SPEC_SYSTEM_PROMPT` — 자연어 → 고정 스키마의 JSON 명세. Few-shot 예시를 포함해서 포맷 이탈을 줄인다.
- `CODER_SYSTEM_PROMPT` — JSON 명세를 받아 파일별 코드 블록(```파일경로 형식) 출력.
- `SINGLE_PASS_SYSTEM_PROMPT` — 명세 단계를 **내부화**한 버전. 중간 분석을 출력하지 말라고 명시. Valves 의 `single_pass=True` 경로에서 사용.
- `DIFF_SPEC_PROMPT` / `DIFF_CODER_PROMPT` — 수정 요청 처리용. 변경된 항목만 담은 diff JSON / 변경된 파일만 출력하는 코드.
- `CHAT_SYSTEM_PROMPT` — 코딩 요청이 아닌 일반 질문용. "참고 자료" 블록이 붙을 경우 출처 표기 규칙까지 명시.

`FRONTEND_FRAMEWORK_RULE` 은 **Vanilla HTML / React(Vite) / Flutter 3가지 중 정확히 하나만 선택**하도록 강제하는 규칙 문자열. 주요 4종의 프롬프트 끝에 concat 되어, 모델이 혼합 구조를 생성하지 못하게 막는다.

---

4. 요청 분류 (코딩 vs 단순 질문)
```

```python
_CODING_ACTION_PATTERNS = (
    "만들어", "만들자", "만들어줘", "만들어봐", "만들래",
    "구현", "짜줘", "짜봐", "짜서", "개발해", "작성해",
    "생성해", "고쳐", "수정해", "리팩터", "리팩토",
    "create ", "build ", "implement ", "write a ", "code a ",
    "refactor", "fix the", "generate a ",
)
_CODING_TECH_KEYWORDS = (
    "html", "css", "javascript", "js", "jsx", "tsx", "react", "vue",
    "svelte", "next.js", "nextjs", "node", "express", "fastapi",
    "flask", "django", "python", "typescript", "flutter", "dart",
    "api", "endpoint", "sql", "database", "dom", "component",
    "컴포넌트", "페이지", "웹사이트", "웹페이지", "웹앱", "앱",
    "대시보드", "로그인 화면", "회원가입", "todo 앱", "계산기",
    "스크립트", "함수", "클래스", "모듈",
)
_SIMPLE_QUESTION_PATTERNS = (
    "뭐야", "뭐지", "무엇", "어떻게", "왜 ", "왜?", "왜야", "이유",
    "차이", "설명", "알려줘", "궁금", "가능해", "가능한", "되나요", "되니",
    "what is", "what's", "how do", "how to", "how does", "why ",
    "difference", "explain", "vs ", "meaning",
)

@classmethod
def _is_simple_question(cls, user_message: str) -> bool:
    if not user_message:
        return False
    msg = user_message.strip().lower()
    # 액션 동사가 있으면 구현 요청 — 단순 질문 아님
    for phrase in cls._CODING_ACTION_PATTERNS:
        if phrase in msg:
            return False
    # 코드 블록이 있으면 기존 코드 수정/분석 요청
    if "```" in user_message:
        return False
    # 물음표로 끝나거나 설명 요청 패턴
    if msg.endswith("?") or msg.endswith("?"):
        return True
    for phrase in cls._SIMPLE_QUESTION_PATTERNS:
        if phrase in msg:
            return True
    # 60자 미만 짧은 메시지 + 구현 동사 없음 → 단순 질문
    if len(msg) < 60 and not any(
        kw in msg for kw in ("만들", "구현", "생성", "작성", "build", "create")
    ):
        return True
    return False

@classmethod
def _is_coding_request(cls, user_message: str) -> bool:
    if not user_message:
        return False
```

```
    if cls._is_simple_question(user_message):
        return False
    msg = user_message.lower()
    # 강한 신호 1: 액션 동사
    for phrase in cls._CODING_ACTION_PATTERNS:
        if phrase in msg:
            return True
    # 강한 신호 2: 코드 블록 포함
    if "```" in user_message:
        return True
    # 약한 신호: 기술 키워드 2개 이상
    tech_hits = sum(1 for kw in cls._CODING_TECH_KEYWORDS if kw in msg)
    if tech_hits >= 2:
        return True
    return False
```

기능

- 3개의 패턴 튜플(액션 동사 / 기술 키워드 / 설명 질문 패턴) 로 휴리스틱 분류.
- `_is_simple_question` — 설명/개념 질문인지 판단. True 면 명세+코드 파이프라인을 건너뛰고 단일 모델로 응답.
- `_is_coding_request` — 코딩 요청인지 판단. False 면 `generate_chat_reply()` 로 빠진다.
- 두 함수의 판단 순서:
    1. 액션 동사(`만들어`, `create` 등) → 무조건 코딩 요청
    2. 백틱 코드 블록 포함 → 코딩 요청 (기존 코드 수정/분석)
    3. 설명 질문 패턴 (`뭐야`, `?`, `how to` 등) → 단순 질문
    4. 기술 키워드 2개 이상 등장 → 코딩 요청
    5. 60자 미만 짧은 메시지 + 구현 동사 없음 → 단순 질문

이 분류는 LLM 호출 전에 수행되므로 비용 없이 경로를 가른다.

---

5. 외부 데이터 수집

실시간 정보(환율, 뉴스, 주가 등)가 필요한 질문은 LLM 이 "제공할 수 없습니다" 로 거절하기 쉽다. 이 모듈은 서버가 직접 웹을 긁어서 **참고 자료 블록**을 프롬프트에 주입한다.

5.1 트리거 판정 & HTML 정제

```
_URL_RE = re.compile(r"https?://[^\s<>\"'`]+")
_REALTIME_TRIGGERS = (
    "지금", "현재", "오늘", "최근", "최신", "방금", "며칠",
    "news", "today", "latest", "current", "price", "시세",
    "환율", "날씨", "주가", "코스피", "코스닥", "비트코인",
    "뉴스", "경기", "스코어", "일정",
)

@classmethod
def _needs_external_data(cls, user_message: str) -> bool:
    if not user_message:
```

```
        return False
    if cls._URL_RE.search(user_message):
        return True
    low = user_message.lower()
    return any(trig.lower() in low for trig in cls._REALTIME_TRIGGERS)

@staticmethod
def _strip_html(html: str, limit: int = 2000) -> str:
    html = re.sub(r"<script[\s\S]*?</script>", " ", html, flags=re.IGNORECASE)
    html = re.sub(r"<style[\s\S]*?</style>",   " ", html, flags=re.IGNORECASE)
    text = re.sub(r"<[^>]+>", " ", html)
    text = (text.replace(" ", " ").replace("&amp;", "&")
                .replace("&lt;", "<").replace("&gt;", ">")
                .replace("&quot;", '"').replace("&#39;", "'"))
    text = re.sub(r"\s+", " ", text).strip()
    return text[:limit]
```

- `_needs_external_data` — 메시지에 URL 이 있거나 실시간 트리거 키워드가 있으면 True.
- `_strip_html` — `<script>`/`<style>` 통째 제거 → 태그 제거 → 엔티티 변환 → 공백 정리 → 길이 컷. 정규식만으로 가볍게 텍스트 추출.

5.2 URL fetch & 카테고리별 직접 소스

```
@classmethod
def _fetch_url(cls, url: str, timeout: float = 8.0) -> str:
    try:
        with httpx.Client(
            timeout=timeout,
            follow_redirects=True,
            headers={"User-Agent": "Mozilla/5.0 (vibe-coding fetcher)"},
        ) as client:
            resp = client.get(url)
            resp.raise_for_status()
            ctype = resp.headers.get("content-type", "")
            body = resp.text
    except Exception as exc:
        return f"[URL 가져오기 실패: {exc}]"
    if "html" in ctype.lower():
        return cls._strip_html(body, limit=2500)
    return body[:2500]

@classmethod
def _fetch_naver_marketindex(cls) -> str:
    """네이버 금융 마켓 인덱스 페이지에서 환율·지수·원자재 스냅샷 추출."""
    try:
        with httpx.Client(
            timeout=8.0,
            follow_redirects=True,
            headers={
                "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                              "AppleWebKit/537.36 (KHTML, like Gecko) "
```

```
                                "Chrome/124.0 Safari/537.36",
            },
        ) as client:
            resp = client.get("https://finance.naver.com/marketindex/")
            resp.encoding = "euc-kr"
            html = resp.text
    except Exception as exc:
        return f"[Naver 금융 마켓인덱스 가져오기 실패: {exc}]"

    # 각 카드: <h3 class="h_lst"><span class="blind">카테고리</span></h3>
    # 내부에 <span class="value">숫자</span>, <span class="change">변동</span>
    items: list[str] = []
    pattern = re.compile(
        r'<h3 class="h_lst"><span class="blind">([^<]+)</span></h3>'
        r'[\s\S]*?<span class="value">([^<]+)</span>'
        r'(?:[\s\S]*?<span class="change">([^<]+)</span>)?',
        re.IGNORECASE,
    )
    for match in pattern.finditer(html):
        cat = cls._strip_html(match.group(1), limit=40)
        value = cls._strip_html(match.group(2), limit=40)
        change = cls._strip_html(match.group(3) or "", limit=40)
        line = f"- {cat}: {value}"
        if change:
            line += f" (변동 {change})"
        items.append(line)
        if len(items) >= 25:
            break

    if not items:
        return "[Naver 금융 마켓인덱스에서 데이터를 추출하지 못했습니다]"
    return (
        "[Naver 금융 마켓인덱스 스냅샷]\n"
        + "\n".join(items)
        + "\n(출처: https://finance.naver.com/marketindex/)"
    )
```

- `_fetch_url` — 단순 GET + HTML 정제. 예외는 문자열로 변환해서 반환(상위에서 그대로 프롬프트에 삽입 가능).
- `_fetch_naver_marketindex` — 환율/코스피/원유/비트코인 등을 **한 번의 요청**으로 전부 가져오기 위한 최적화 경로. `euc-kr` 인코딩 강제가 포인트. 정규식으로 `h_lst` / `value` / `change` 3 필드를 한 번에 뽑고, 파싱 실패 시 안내 문구로 폴백.

5.3 검색 엔진 폴백

```
@classmethod
def _fetch_naver_search(cls, query: str, timeout: float = 8.0) -> list[dict[str,
str]]:
    """Naver 통합 검색 HTML 결과에서 상위 스니펫 추출."""
    results: list[dict[str, str]] = []
    try:
```

```
        with httpx.Client(
            timeout=timeout,
            follow_redirects=True,
            headers={
                "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                              "AppleWebKit/537.36 (KHTML, like Gecko) "
                              "Chrome/124.0 Safari/537.36",
                "Accept-Language": "ko,en;q=0.8",
            },
        ) as client:
            resp = client.get(
                "https://search.naver.com/search.naver",
                params={"query": query},
            )
            resp.raise_for_status()
            html = resp.text
    except Exception:
        return results

    # Naver 검색 결과 카드: <a ... class="...news_tit..." href="..." title="...">
및
    # <div class="...desc..."> 등 클래스가 자주 바뀌므로 대강의 링크 + 텍스트만 뽑음.
    for match in re.finditer(
        r'<a[^>]+href="(https?://[^"]+)"[^>]*title="([^"]+)"',
        html,
    ):
        url = match.group(1)
        title = cls._strip_html(match.group(2), limit=120)
        if url.startswith("https://search.naver.com"):
            continue
        if any(r["url"] == url for r in results):
            continue
        results.append({"title": title, "snippet": "", "url": url})
        if len(results) >= 5:
            break

    # 스니펫 텍스트 — 일반적인 본문 스니펫 클래스 패턴 수집
    snippets: list[str] = []
    for match in re.finditer(
        r'<(?:a|div|span)[^>]+class="[^"]*
(?:api_txt_lines|dsc_txt_wrap|total_dsc|news_dsc)[^"]*"[^>]*>([\s\S]*?)
</(?:a|div|span)>',
        html,
    ):
        text = cls._strip_html(match.group(1), limit=300)
        if text and len(text) > 20:
            snippets.append(text)
        if len(snippets) >= 5:
            break

    for i, r in enumerate(results):
        if i < len(snippets):
            r["snippet"] = snippets[i]
    return results
```

```python
@classmethod
def _duckduckgo_html_search(cls, query: str, timeout: float = 8.0) ->
list[dict[str, str]]:
    """DuckDuckGo HTML 엔드포인트 폴백."""
    results: list[dict[str, str]] = []
    try:
        with httpx.Client(
            timeout=timeout,
            follow_redirects=True,
            headers={"User-Agent": "Mozilla/5.0 (vibe-coding fetcher)"},
        ) as client:
            resp = client.get(
                "https://html.duckduckgo.com/html/",
                params={"q": query},
            )
            resp.raise_for_status()
            html = resp.text
    except Exception:
        return results

    pattern = re.compile(
        r'<a[^>]*class="result__a"[^>]*href="([^"]+)"[^>]*>([^<]+)</a>'
        r'[\s\S]*?class="result__snippet"[^>]*>([\s\S]*?)</a>',
        re.IGNORECASE,
    )
    for match in pattern.finditer(html):
        if len(results) >= 5:
            break
        url = match.group(1)
        title = cls._strip_html(match.group(2), limit=120)
        snippet = cls._strip_html(match.group(3), limit=300)
        results.append({"title": title, "snippet": snippet, "url": url})
    return results
```

- `_fetch_naver_search` — 링크 추출과 스니펫 추출을 **두 단계로 나눠** 수행. Naver 의 결과 카드 클래스명이 자주 바뀌기 때문에, 제목은 `title` 속성으로 확실히 잡고 스니펫은 대표적인 4개 클래스 패턴 (`api_txt_lines` / `dsc_txt_wrap` / `total_dsc` / `news_dsc`) 중 하나에 걸리는 것을 수집한 뒤 순서대로 매칭.
- `_duckduckgo_html_search` — 단일 정규식으로 `result__a` (제목+URL) 와 `result__snippet` (본문)을 한 번에 잡는다. 구조가 단순해서 Naver 보다 파싱이 훨씬 쉽다.
- 두 함수 모두 어떤 이유로든 실패하면 **빈 리스트** 를 반환하는 동일 규약. 상위 `fetch_external_context` 가 하나 실패하면 다른 하나로 폴백할 수 있도록 인터페이스를 맞춰 놨다.

5.4 통합 컨텍스트 빌더 `fetch_external_context`

```python
@classmethod
def fetch_external_context(cls, user_message: str) -> str:
    if not cls._needs_external_data(user_message):
        return ""
```

```
    parts: list[str] = []
    msg_low = user_message.lower()

    # 1) 메시지 안의 URL 직접 fetch (최대 2개)
    urls = cls._URL_RE.findall(user_message)[:2]
    for url in urls:
        parts.append(f"[페이지: {url}]\n{cls._fetch_url(url)}")

    if not urls:
        # 2) 금융 키워드 → Naver 금융 마켓인덱스 스냅샷
        finance_kws = (
            "환율", "달러", "엔화", "유로", "위안",
            "코스피", "kospi", "코스닥", "kosdaq",
            "다우", "나스닥", "s&p",
            "금값", "유가", "wti", "비트코인", "bitcoin", "이더리움", "ethereum",
            "원/달러", "exchange rate",
        )
        if any(kw in msg_low for kw in finance_kws):
            parts.append(cls._fetch_naver_marketindex())

        # 3) 그래도 비었으면 Naver 검색 → 실패 시 DDG
        if not parts:
            results = cls._fetch_naver_search(user_message)
            source = "Naver 검색"
            if not results:
                results = cls._duckduckgo_html_search(user_message)
                source = "DuckDuckGo"
            if results:
                # 상위 결과 1개는 본문까지 fetch 해서 깊게 읽기
                top = results[0]
                top_body = cls._fetch_url(top["url"])
                lines = [f"[{source} 상위 결과]"]
                for i, r in enumerate(results, 1):
                    lines.append(f"{i}. {r['title']}\n   {r['snippet']}\n
({r['url']})")
                lines.append("")
                lines.append(f"[상위 결과 본문 발췌: {top['url']}]")
                lines.append(top_body)
                parts.append("\n".join(lines))

    if not parts:
        return ""

    return "--- 참고 자료 ---\n" + "\n\n".join(parts) + "\n--- /참고 자료 ---"
```

기능

- 3단계 우선순위:
  1. 메시지에 URL 이 박혀 있으면 그걸 직접 fetch.
  2. URL 이 없고 금융 키워드가 있으면 Naver 마켓인덱스 스냅샷.
  3. 둘 다 아니면 Naver 검색 → DDG 폴백 + 상위 1건 본문까지 fetch.

- 최종 결과는 `--- 참고 자료 ---` **블록** 으로 감싼 단일 문자열. 상위 호출부에서 이 블록을 그대로 프롬프트에 주입한다.

---

6. 대화 히스토리 정규화

```
_REF_BLOCK_RE = re.compile(
    r"\n*---\s*참고 자료\s*---[\s\S]*?---\s*/참고 자료\s*---\n*",
    re.IGNORECASE,
)
_REF_TAIL_RE = re.compile(
    r"\n*위 '참고 자료'[\s\S]*?거절 문구[\s\S]*?금지\.?\s*$",
)
_FETCH_STATUS_RE = re.compile(r"^🌐 외부 데이터 수집 중\.{0,3}\s*", re.MULTILINE)

_HISTORY_MAX_TURNS      = 8      # 최근 8개 메시지(=약 4왕복)만
_HISTORY_PER_MSG_CHARS  = 1500   # 메시지 1개당 최대 글자수
_HISTORY_TOTAL_CHARS    = 6000   # 히스토리 전체 최대 글자수

@classmethod
def _normalize_chat_messages(
    cls,
    messages: list[dict[str, Any]] | None,
) -> list[dict[str, str]]:
    normalized: list[dict[str, str]] = []
    if not messages:
        return normalized

    for raw in messages[-cls._HISTORY_MAX_TURNS:]:
        if not isinstance(raw, dict):
            continue
        role = raw.get("role")
        if role not in {"user", "assistant"}:
            continue
        content = raw.get("content")
        if content is None:
            continue
        text = str(content)
        # 이전 턴에 주입됐던 잔재 제거
        text = cls._REF_BLOCK_RE.sub("\n", text)
        text = cls._REF_TAIL_RE.sub("", text)
        text = cls._FETCH_STATUS_RE.sub("", text)
        text = text.strip()
        if not text:
            continue
        # 메시지 1개가 너무 길면 앞/뒤만 남기고 중간 생략
        if len(text) > cls._HISTORY_PER_MSG_CHARS:
            head = text[: cls._HISTORY_PER_MSG_CHARS // 2]
            tail = text[-cls._HISTORY_PER_MSG_CHARS // 2 :]
            text = f"{head}\n...(중략)...\n{tail}"
        normalized.append({"role": role, "content": text})
```

```
    # 전체 글자수 제한 — 뒤(최신)부터 누적해서 넘으면 앞(오래된) 쪽을 버림
    total = 0
    kept_reversed: list[dict[str, str]] = []
    for msg in reversed(normalized):
        total += len(msg["content"])
        if total > cls._HISTORY_TOTAL_CHARS and kept_reversed:
            break
        kept_reversed.append(msg)
    return list(reversed(kept_reversed))

def _build_model_messages(
    self,
    system: str,
    user: str = "",
    messages: list[dict[str, Any]] | None = None,
) -> list[dict[str, str]]:
    conversation = self._normalize_chat_messages(messages)
    user = (user or "").strip()
    if user:
        if not conversation or conversation[-1]["role"] != "user" or
conversation[-1]["content"] != user:
            conversation.append({"role": "user", "content": user})
    return [{"role": "system", "content": system}, *conversation]
```

기능

- 3개의 정규식으로 **이전 턴에 주입됐던 것들을 제거**: 참고 자료 블록 / 참고 자료 사용 지시문 / "🌐 외부 데이터 수집 중" 상태 라인. 이걸 안 지우면 매 턴마다 누적돼서 프롬프트가 폭증한다.
- `_normalize_chat_messages` 는 3단계로 길이를 제한한다:
  1. 최근 `_HISTORY_MAX_TURNS = 8` 개만 남김
  2. 메시지 1개가 `_HISTORY_PER_MSG_CHARS = 1500` 자 넘으면 앞/뒤 절반만 남기고 중간 생략
  3. 총합이 `_HISTORY_TOTAL_CHARS = 6000` 자 넘으면 뒤(최신)부터 누적해서 초과하는 순간 앞쪽 컷
- `_build_model_messages` — 시스템 + 정규화된 히스토리 + (중복이 아니면) 현재 유저 메시지를 붙여 Ollama 가 원하는 포맷(`[{role, content}, ...]`)으로 만들어 반환.

---

7. 일반 채팅 응답 `generate_chat_reply`

```
def generate_chat_reply(
    self, user_message: str, messages: list[dict[str, Any]] | None = None
) -> Generator[str, None, None]:
    """일반 질문에 대한 간단한 채팅 응답 (명세/코드 단계 없이 1-pass)."""
    missing = self._missing_config()
    if missing:
        yield "설정이 비어 있습니다: " + ", ".join(missing)
        return

    _, coder_model = self._select_models(user_message)

    system_prompt = self.CHAT_SYSTEM_PROMPT
```

```
    external: str = ""
    if self._needs_external_data(user_message):
        yield "🌐 외부 데이터 수집 중...\n\n"
        try:
            external = self.fetch_external_context(user_message)
        except Exception as exc:
            external = ""
            yield f"(외부 데이터 수집 실패: {exc})\n\n"

    effective_user = user_message
    if external:
        system_prompt = (
            f"{self.CHAT_SYSTEM_PROMPT}\n\n"
            "[규칙] 사용자 메시지에 '참고 자료' 블록이 포함되어 있다면, "
            "이는 방금 서버가 실제 웹에서 가져온 최신 수치/가격/지수/뉴스입니다. "
            "절대로 '실시간 정보를 제공할 수 없습니다' 같은 거절 문구를 쓰지 마십시오.
"
            "반드시 자료 안의 수치를 직접 인용해 답하고, "
            "답변 끝에 (출처: URL) 형태로 1~2개 출처를 덧붙이세요."
        )
        effective_user = (
            f"{user_message}\n\n{external}\n\n"
            "위 '참고 자료' 안의 수치를 사용해서 위 질문에 한국어로 바로 답하세요. "
            "거절 문구(예: '실시간 정보 제공 불가') 금지."
        )

    try:
        # 외부 데이터가 있을 때는 이전 대화 맥락을 빼고 단발성으로 답하게 함
        effective_messages = None if external else messages
        for tok in self._stream_ollama_sync(
            model=coder_model,
            system=system_prompt,
            user=effective_user,
            messages=effective_messages,
        ):
            yield tok
    except Exception as exc:
        yield self._format_model_error("응답 생성", exc)
        return
```

기능

- 코딩 요청이 아닌 일반 질문 경로. Spec 모델을 안 거치고 Coder 모델 하나로 바로 답한다 (reuse).
- `_needs_external_data` 가 True 면 웹 크롤링 → 참고 자료 블록 주입.
- **중요 포인트**: 참고 자료가 있을 때는 `messages` 를 `None` 으로 넘겨 **이전 대화 히스토리를 무시**한다. 과거 턴에 "실시간 정보 제공 불가" 같은 거절 답변이 있으면 모델이 같은 거절을 반복하는 경향이 있기 때문.
- 에러는 `_format_model_error` 로 사용자 친화적인 한국어 문구로 변환.

---

8. 자동 라우팅 (복잡도 추정)

```python
@staticmethod
def _estimate_complexity(user_message: str) -> str:
    """Estimate request complexity: 'simple' or 'complex'."""
    indicators = [
        "데이터베이스", "DB", "인증", "로그인", "API", "백엔드",
        "서버", "배포", "Docker", "Next.js", "React", "라우팅",
        "상태 관리", "Redux", "database", "authentication",
        "WebSocket", "실시간", "대시보드", "관리자",
    ]
    message_lower = user_message.lower()
    hits = sum(1 for ind in indicators if ind.lower() in message_lower)
    word_count = len(user_message.split())
    if hits >= 2 or word_count > 80:
        return "complex"
    return "simple"

def _select_models(self, user_message: str) -> tuple[str, str]:
    """Return (spec_model, coder_model) based on auto-routing."""
    if not self.valves.auto_route:
        return self.valves.spec_model, self.valves.coder_model

    complexity = self._estimate_complexity(user_message)
    if complexity == "simple" and self.valves.spec_model_light and
self.valves.coder_model_light:
        return self.valves.spec_model_light, self.valves.coder_model_light
    return self.valves.spec_model, self.valves.coder_model
```

기능

- `_estimate_complexity` — 복잡도 지표 키워드(DB, 인증, API, WebSocket, 대시보드 등) 2개 이상이거나 단어 수 80 개 초과면 `"complex"`, 아니면 `"simple"`.
- `_select_models` — `auto_route=False` 면 그냥 본 모델. `True` 이고 경량 모델 2종이 다 설정돼 있고 요청이 simple 이면 경량 모델 쌍 반환. 그 외는 본 모델.
- 리턴이 항상 `(spec_model, coder_model)` 튜플이어서, 상위 호출부는 auto_route 여부를 몰라도 된다.

---

9. 메인 파이프 `pipe`

```python
def pipe(
    self,
    user_message: str,
    model_id: str | None = None,
    messages: list[dict[str, Any]] | None = None,
    body: dict[str, Any] | None = None,
    username: str = "anonymous",
    chat_id: str | None = None,
) -> Generator[str, None, None]:
    del model_id, body

    missing = self._missing_config()
```

```
        if missing:
            yield (
                "설정이 비어 있습니다. `pipelines/config.json` 에서 "
                f"{', '.join(missing)} 값을 입력한 뒤 다시 실행해 주세요."
            )
            return

        # 코딩 요청이 아니면 명세/코드 단계를 건너뛰고 바로 채팅 응답
        if not self._is_coding_request(user_message):
            yield from self.generate_chat_reply(user_message, messages=messages)
            return

        spec_model, coder_model = self._select_models(user_message)

        # ── single_pass: 1회 호출로 스트리밍 ──────────────────────────
        if self.valves.single_pass:
            buf = ""
            try:
                for tok in self._stream_ollama_sync(
                    model=coder_model,
                    system=self.SINGLE_PASS_SYSTEM_PROMPT,
                    user=user_message,
                    messages=messages,
                ):
                    buf += tok
                    if "\n" in tok:
                        yield buf
                        buf = ""
            except Exception as exc:
                if buf:
                    yield buf
                yield self._format_model_error("코드 생성", exc)
                return
            if buf:
                yield buf
            return

        # ── dual: Plan 모델(JSON 명세) → Coder 모델(스트리밍) ──────────
        # keep_alive=0 : VRAM 절약을 위해 spec 생성 직후 plan 모델을 언로드
        yield "📝 명세서 작성 중... (plan 모델)\n\n"
        try:
            spec_response = self._run_async(self._call_model(
                model=spec_model,
                system=self.SPEC_SYSTEM_PROMPT,
                user=user_message,
                messages=messages,
                keep_alive=0,
            ))
        except Exception as exc:
            yield self._format_model_error("요청 분석", exc)
            return

        try:
            spec_json = self._parse_json(spec_response)
```

```
            self._validate_spec(spec_json)
        except ValueError as exc:
            yield (
                "요청 분석 결과를 처리하지 못했습니다: "
                f"{exc}\n\n원본 응답:\n```\n{spec_response[:2000]}\n```"
            )
            return

        # 사용자별 명세서 저장 (실패해도 파이프라인은 계속)
        try:
            saved_path = self.save_spec(
                username=username, chat_id=chat_id,
                spec=spec_json, user_message=user_message,
            )
            yield f"✅ 명세서 저장 완료:
`specs/{saved_path.parent.name}/{saved_path.name}`\n\n"
        except OSError as exc:
            yield f"⚠️ 명세서 저장 실패(무시하고 계속 진행): {exc}\n\n"

        # 명세서 미리보기 (접힌 형태)
        spec_preview = json.dumps(spec_json, ensure_ascii=False, indent=2)
        yield (
            "<details><summary>📋 생성된 명세서 (클릭하여 펼치기)</summary>\n\n"
            f"```json\n{spec_preview}\n```\n\n</details>\n\n"
        )

        # Coder 모델: 명세서 기반 코드 생성 (실시간 스트리밍)
        yield "💻 코드 생성 중... (coder 모델)\n\n"
        buf = ""
        try:
            for tok in self._stream_ollama_sync(
                model=coder_model,
                system=self.CODER_SYSTEM_PROMPT,
                user=(
                    "다음 기술 명세서를 기반으로 코드를 생성하세요:\n"
                    f"{json.dumps(spec_json, ensure_ascii=False, indent=2)}"
                ),
                messages=messages,
            ):
                buf += tok
                if "\n" in tok:
                    yield buf
                    buf = ""
        except Exception as exc:
            if buf:
                yield buf
            yield self._format_model_error("코드 생성", exc)
            return
        if buf:
            yield buf
```

기능

Pipeline 클래스의 메인 엔트리. 상태 흐름은 다음 4가지 분기 중 하나를 탄다:

1. **설정 누락** — `_missing_config()` 결과가 있으면 안내 문구만 yield 하고 종료.
2. **일반 채팅** — `_is_coding_request()` 가 False 면 `generate_chat_reply()` 로 위임.
3. **single_pass** — Coder 모델 1회 호출로 끝. 토큰을 모아서 개행(`\n`) 단위로 flush → 스트리밍 체감 속도 확보.
4. **dual (Plan → Code)** —
   - Plan 모델을 `keep_alive=0` 로 호출. 응답 직후 VRAM 에서 언로드되어 Coder 모델이 로드될 자리를 확보한다 (로컬 GPU 에서 중요).
   - JSON 파싱 + `_validate_spec` 으로 스키마 검증. 실패 시 원본 응답 일부를 사용자에게 보여주고 종료.
   - `save_spec` 으로 디스크에 저장. 저장 실패(`OSError`)는 경고만 내고 진행 — 저장 실패가 코드 생성을 막지 않도록 격리.
   - 생성된 명세서를 `<details>` 로 접어 미리보기로 노출.
   - Coder 모델을 **실시간 스트리밍**으로 호출. 역시 개행 단위 flush.

모든 LLM 호출은 try/except 로 감싸서 `_format_model_error` 로 변환 후 yield.

---

10. Two-stage 모드 `generate_spec` / `generate_code_from_spec`

```
def generate_spec(
    self,
    user_message: str,
    messages: list[dict[str, Any]] | None = None,
    username: str = "anonymous",
    chat_id: str | None = None,
) -> dict[str, Any]:
    """사용자 요청 → JSON 명세만 생성하여 저장한 뒤 dict로 반환."""
    missing = self._missing_config()
    if missing:
        raise RuntimeError("설정이 비어 있습니다: " + ", ".join(missing))

    spec_model, _ = self._select_models(user_message)
    spec_response = self._run_async(self._call_model(
        model=spec_model,
        system=self.SPEC_SYSTEM_PROMPT,
        user=user_message,
        messages=messages,
        keep_alive=0,
    ))

    try:
        spec_json = self._parse_json(spec_response)
        self._validate_spec(spec_json)
    except ValueError as exc:
        raise ValueError(
            f"명세서 파싱 실패: {exc}\n원본 응답:\n{spec_response[:2000]}"
        ) from exc

    self.save_spec(
```

```
            username=username, chat_id=chat_id,
            spec=spec_json, user_message=user_message,
        )
        return spec_json


    def generate_code_from_spec(
        self,
        spec: dict[str, Any],
        user_message: str = "",
        messages: list[dict[str, Any]] | None = None,
        username: str = "anonymous",
        chat_id: str | None = None,
    ) -> Generator[str, None, None]:
        """주어진 (사용자 검토/수정된) 명세서 → 코드 스트림."""
        del username, chat_id
        missing = self._missing_config()
        if missing:
            yield "설정이 비어 있습니다: " + ", ".join(missing)
            return

        try:
            self._validate_spec(spec)
        except ValueError as exc:
            yield f"⚠ 명세서 검증 실패: {exc}"
            return

        _, coder_model = self._select_models(user_message or "")
        buffer = ""
        try:
            for tok in self._stream_ollama_sync(
                model=coder_model,
                system=self.CODER_SYSTEM_PROMPT,
                user=(
                    "다음 기술 명세서를 기반으로 코드를 생성하세요:\n"
                    f"{json.dumps(spec, ensure_ascii=False, indent=2)}"
                ),
                messages=messages,
            ):
                buffer += tok
                if "\n" in tok:
                    yield buffer
                    buffer = ""
        except Exception as exc:
            if buffer:
                yield buffer
            yield self._format_model_error("코드 생성", exc)
            return
        if buffer:
            yield buffer
```

기능

`pipe()` 의 dual 경로를 **두 단계로 분리**한 버전. "확인 후 진행" 워크플로에서 사용한다.

- `generate_spec` — 명세 JSON 만 생성/검증/저장하고 **dict 로 반환**. 제너레이터가 아니다. 호출부가 이 dict 를 사용자에게 보여주고 수정 후 2단계로 넘긴다.
- `generate_code_from_spec` — 이미 확정된 명세를 받아서 코드만 스트리밍. 명세 검증은 다시 수행(호출부에서 수정됐을 수 있으므로).
- 예외 처리 스타일이 다르다: `generate_spec` 은 `raise`, `generate_code_from_spec` 은 제너레이터라서 `yield` 로 에러 메시지 반환.

---

11. Diff 수정 파이프라인 `pipe_modify`

```
def pipe_modify(
    self,
    modification_request: str,
    original_spec: dict[str, Any],
    existing_files: dict[str, str],
    project_id: str = "",
) -> Generator[str, None, None]:
    """Handle modification requests via diff spec pipeline."""
    del project_id
    spec_model, coder_model = self._select_models(modification_request)

    diff_input = (
        f"원본 명세서:\n{json.dumps(original_spec, ensure_ascii=False,
indent=2)}\n\n"
        f"사용자 수정 요청:\n{modification_request}"
    )

    try:
        diff_response = self._run_async(self._call_model(
            model=spec_model,
            system=self.DIFF_SPEC_PROMPT,
            user=diff_input,
        ))
    except Exception as exc:
        yield self._format_model_error("수정 요청 분석", exc)
        return

    try:
        diff_json = self._parse_json(diff_response)
        self._validate_diff_spec(diff_json)
    except ValueError as exc:
        yield f"수정 요청 분석 결과를 처리하지 못했습니다: {exc}"
        return

    # 기존 파일들을 코드 블록 형태로 이어붙임
    existing_code_text = ""
    for path, content in existing_files.items():
        existing_code_text += f"```{path}\n{content}\n```\n\n"

    coder_input = (
```

```
            f"기존 코드:\n{existing_code_text}\n"
            f"diff 명세서:\n{json.dumps(diff_json, ensure_ascii=False, indent=2)}\n\n"
            f"위 diff 명세서에 명시된 변경 사항만 반영하여 수정된 파일만 출력하세요."
        )

        try:
            code_response = self._run_async(self._call_model(
                model=coder_model,
                system=self.DIFF_CODER_PROMPT,
                user=coder_input,
            ))
        except Exception as exc:
            yield self._format_model_error("코드 수정", exc)
            return

        # Open WebUI 에서 ```파일경로 형식이 깨지는 문제 수정
        yield from self._stream_text(self._format_code_for_webui(code_response))
```

기능

수정 요청 전용 파이프. 일반 `pipe()` 와 두 가지가 다르다:

1. **Diff 명세**(`DIFF_SPEC_PROMPT`) 를 먼저 뽑는다. 변경 항목만 담긴 작은 JSON 이라 전체 명세 재생성보다 훨씬 싸다.
2. 코드 생성도 **변경된 파일만** 출력하도록 `DIFF_CODER_PROMPT` 를 쓰고, 기존 코드를 컨텍스트에 포함시킨다.

스트리밍을 쓰지 않고 `_call_model` 로 전체 응답을 받은 뒤 `_format_code_for_webui` 로 포맷을 고쳐서 `_stream_text` 로 쪼개 yield 한다. 수정 응답은 보통 짧아서 실시간 토큰 스트리밍의 이득이 적기 때문.

---

12. 설정/명세 검증

```
def _missing_config(self) -> list[str]:
    required = {
        "spec_model":  self.valves.spec_model,
        "coder_model": self.valves.coder_model,
        "ollama_url":  self.valves.ollama_url,
    }
    return [name for name, value in required.items() if not value.strip()]

@staticmethod
def _validate_spec(spec: dict[str, Any]) -> None:
    required = {"project", "files", "components", "api", "constraints",
"user_story"}
    missing = sorted(required - set(spec))
    if missing:
        raise ValueError(f"Missing required keys: {', '.join(missing)}")
    if not isinstance(spec["files"], list) or not spec["files"]:
        raise ValueError("files must be a non-empty list")
    for item in spec["files"]:
```

```
        if not isinstance(item, dict) or not item.get("path"):
            raise ValueError("each file item must include a path")
    project = spec.get("project", {})
    if not isinstance(project, dict):
        raise ValueError("project must be an object")
    for key in ("name", "type", "scope", "tech_stack", "description"):
        if key not in project:
            raise ValueError(f"project.{key} is required")
    allowed_scopes = {"frontend", "backend", "fullstack", "cli", "data",
"general"}
    if project.get("scope") not in allowed_scopes:
        raise ValueError(
            f"project.scope must be one of: {', '.join(sorted(allowed_scopes))}"
        )

@staticmethod
def _validate_diff_spec(diff: dict[str, Any]) -> None:
    allowed_keys = {"modified_files", "new_constraints", "removed_components"}
    if not any(key in diff for key in allowed_keys):
        raise ValueError(
            f"diff spec must contain at least one of: {',
'.join(sorted(allowed_keys))}"
        )
    if "modified_files" in diff:
        if not isinstance(diff["modified_files"], list):
            raise ValueError("modified_files must be a list")
        for item in diff["modified_files"]:
            if not isinstance(item, dict) or not item.get("path"):
                raise ValueError("each modified_file must have a path")
```

기능

- `_missing_config` — Valves 의 3개 필수 필드(`spec_model`/`coder_model`/`ollama_url`) 중 공백인 항목의 이름 리스트 반환. 파이프라인 시작 시 게이트로 사용.
- `_validate_spec` — SPEC 스키마의 모든 필수 키, `files` 배열의 path, `project` 객체의 하위 키, `project.scope` 의 허용 값 집합까지 검사. 실패 시 `ValueError` 로 원인 메시지.
- `_validate_diff_spec` — DIFF 스키마는 더 느슨하다. 허용된 3개 키 중 **최소 1개만 있으면 통과**. `modified_files` 가 있을 때만 각 항목의 `path` 를 요구.

LLM 출력이 스키마를 벗어났을 때 파이프라인이 잘못된 데이터로 다음 단계에 진입하는 것을 막는 안전망 역할.

---

13. Ollama 호출 (비동기 ↔ 동기 브리지)

13.1 `_run_async` — 기존 이벤트 루프 안에서 코루틴 실행

```
@staticmethod
def _run_async(coro: Any) -> Any:
    """Run a coroutine safely from a synchronous context.
```

```
    Open WebUI may call ``pipe`` from within an already-running event loop.
    In that case ``asyncio.run()`` raises RuntimeError, so we fall back to
    executing the coroutine in a dedicated daemon thread with its own loop.
    """
    try:
        asyncio.get_running_loop()
        # 이미 루프가 돌고 있음 — 새 스레드에서 별도 루프로 실행
        result_holder: list[Any] = []
        exc_holder: list[BaseException] = []

        def _run() -> None:
            try:
                result_holder.append(asyncio.run(coro))
            except BaseException as exc:
                exc_holder.append(exc)

        t = threading.Thread(target=_run, daemon=True)
        t.start()
        t.join()
        if exc_holder:
            raise exc_holder[0]
        return result_holder[0]
    except RuntimeError:
        # 현재 스레드에 루프 없음
        return asyncio.run(coro)
```

기능

Open WebUI 는 이미 이벤트 루프 안에서 `pipe()` 를 호출하기 때문에, `asyncio.run()` 을 직접 쓰면 `RuntimeError: asyncio.run() cannot be called from a running event loop` 가 난다. 이 헬퍼는:

- 현재 스레드에 루프가 있으면 → **전용 데몬 스레드**에서 `asyncio.run()` 으로 실행하고 `join()` 으로 블로킹 대기. 결과/예외는 리스트로 주고받는다.
- 루프가 없으면 → 그냥 `asyncio.run()`.

13.2 `_call_ollama` — 논스트리밍 완성 응답

```
    async def _call_model(
        self, model, system, user,
        messages=None, keep_alive=None,
    ) -> str:
        return await self._call_ollama(
            model=model, system=system, user=user,
            messages=messages, keep_alive=keep_alive,
        )

    async def _call_ollama(
        self,
        model: str,
        system: str,
        user: str,
```

```
    messages: list[dict[str, Any]] | None = None,
    keep_alive: Any = None,
) -> str:
    request_body: dict[str, Any] = {
        "model": model,
        "stream": False,
        "messages": self._build_model_messages(
            system=system, user=user, messages=messages,
        ),
    }
    if keep_alive is not None:
        request_body["keep_alive"] = keep_alive

    async with httpx.AsyncClient(timeout=600) as client:
        response = await client.post(
            f"{self.valves.ollama_url.rstrip('/')}/api/chat",
            json=request_body,
        )
        response.raise_for_status()
        payload = response.json()

    try:
        return payload["message"]["content"]
    except KeyError as exc:
        raise RuntimeError(f"Unexpected Ollama response: {payload}") from exc
```

기능

- `/api/chat` 엔드포인트를 **논스트림 모드**로 호출. 전체 응답을 받아서 `message.content` 한 줄을 리턴.
- `keep_alive` 는 Ollama 의 모델 메모리 유지 시간. `0` 을 넘기면 응답 후 즉시 언로드 → VRAM 확보. `pipe()` 의 dual 경로에서 Spec 모델에만 이 값을 쓴다.
- 파싱 실패는 전체 payload 를 담아 `RuntimeError` 로 던져서 디버깅 단서 확보.
- 타임아웃 600 초는 로컬 대형 모델의 긴 첫 토큰 지연을 감안한 값.

13.3 `_stream_ollama_sync` — 실시간 토큰 스트리밍

```
def _stream_ollama_sync(
    self,
    model: str,
    system: str,
    user: str,
    messages: list[dict[str, Any]] | None = None,
    keep_alive: Any = None,
) -> Generator[str, None, None]:
    """Ollama `/api/chat` 를 stream=True 로 호출하여 토큰을 바로 yield."""
    import queue

    request_body: dict[str, Any] = {
        "model": model,
        "stream": True,
        "messages": self._build_model_messages(
```

```
            system=system, user=user, messages=messages,
        ),
    }
    if keep_alive is not None:
        request_body["keep_alive"] = keep_alive

    q: "queue.Queue[tuple[str, Any]]" = queue.Queue(maxsize=64)
    url = f"{self.valves.ollama_url.rstrip('/')}/api/chat"

    async def _producer() -> None:
        try:
            async with httpx.AsyncClient(timeout=600) as client:
                async with client.stream("POST", url, json=request_body) as resp:
                    resp.raise_for_status()
                    async for line in resp.aiter_lines():
                        if not line:
                            continue
                        try:
                            data = json.loads(line)
                        except json.JSONDecodeError:
                            continue
                        chunk = (data.get("message") or {}).get("content") or ""
                        if chunk:
                            q.put(("chunk", chunk))
                        if data.get("done"):
                            break
        except Exception as exc:
            q.put(("error", exc))
        finally:
            q.put(("done", None))

    def _run_loop() -> None:
        try:
            asyncio.run(_producer())
        except Exception as exc:
            q.put(("error", exc))
            q.put(("done", None))

    t = threading.Thread(target=_run_loop, daemon=True)
    t.start()
    while True:
        kind, value = q.get()
        if kind == "chunk":
            yield value
        elif kind == "error":
            raise value  # type: ignore[misc]
        else:  # done
            break
```

기능

- Ollama 의 `stream=True` NDJSON 응답을 읽는 **비동기 producer** 를 스레드로 띄우고, 메인 스레드는 `queue.Queue` 에서 꺼내 동기 제너레이터로 yield.

- 큐 아이템은 `("chunk" | "error" | "done", value)` 형태의 태그드 튜플 — 에러도 큐로 흘려서 예외가 스레드 경계에서 소실되지 않도록 처리.
- `maxsize=64` 는 생산자가 너무 빠를 때 소비자 쪽이 밀리지 않도록 하는 backpressure.
- `_run_async` 와 달리, 이쪽은 **제너레이터**가 필요하므로 `thread.join()` 으로 기다릴 수 없어서 큐 기반 브리지가 필수.

---

14. 명세 저장 & 로드

14.1 경로/이름 헬퍼

```
@staticmethod
def _sanitize_path_component(value: str) -> str:
    cleaned = re.sub(r"[^A-Za-z0-9_\-]+", "_", value.strip())
    return cleaned.strip("._") or "default"

@classmethod
def _spec_dir(cls, username: str) -> Path:
    safe = cls._sanitize_path_component(username or "anonymous")
    path = SPECS_ROOT / safe
    path.mkdir(parents=True, exist_ok=True)
    return path

_SPEC_DAYFILE_RE = re.compile(r"^\d{8}\.jsonl$")

@classmethod
def _spec_dayfile(cls, username: str, when: datetime | None = None) -> Path:
    d = cls._spec_dir(username)
    return d / ((when or datetime.now()).strftime("%Y%m%d") + ".jsonl")
```

- `_sanitize_path_component` — 알파뉴메릭/언더스코어/하이픈 외 문자는 모두 `_` 로. 빈 결과는 `"default"` 로 폴백. 경로 주입 방지.
- `_spec_dir` — `specs/<sanitized_username>/` 을 보장 생성.
- `_spec_dayfile` — `YYYYMMDD.jsonl` 형태의 그날 파일 경로.

14.2 키 순서 정렬

```
_SPEC_KEY_ORDER = (
    "project", "files", "components", "data_model",
    "api", "logic", "styling", "constraints", "notes",
)
_RECORD_KEY_ORDER = (
    "saved_at", "chat_id", "user_message", "spec",
)

@classmethod
def _ordered_spec(cls, spec: dict[str, Any]) -> dict[str, Any]:
    if not isinstance(spec, dict):
        return spec
```

```
    ordered: dict[str, Any] = {}
    for k in cls._SPEC_KEY_ORDER:
        if k in spec:
            ordered[k] = spec[k]
    for k, v in spec.items():
        if k not in ordered:
            ordered[k] = v
    return ordered

@classmethod
def _ordered_record(cls, record: dict[str, Any]) -> dict[str, Any]:
    ordered: dict[str, Any] = {}
    for k in cls._RECORD_KEY_ORDER:
        if k in record:
            ordered[k] = record[k]
    for k, v in record.items():
        if k not in ordered:
            ordered[k] = v
    if isinstance(ordered.get("spec"), dict):
        ordered["spec"] = cls._ordered_spec(ordered["spec"])
    return ordered
```

- 지정 순서대로 먼저 넣고, 나머지는 뒤에 append. 스키마에 새 키가 추가돼도 깨지지 않는다.
- 디스크에 저장될 때 항상 같은 키 순서 → **git diff 가 깔끔**하게 나옴.

14.3 JSONL I/O

```
@staticmethod
def _read_jsonl(path: Path) -> list[dict[str, Any]]:
    if not path.exists():
        return []
    out: list[dict[str, Any]] = []
    try:
        for line in path.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line:
                continue
            try:
                obj = json.loads(line)
            except json.JSONDecodeError:
                continue
            if isinstance(obj, dict):
                out.append(obj)
    except OSError:
        return []
    return out

@staticmethod
def _write_jsonl(path: Path, records: list[dict[str, Any]]) -> None:
    lines = [
        json.dumps(r, ensure_ascii=False, separators=(",", ":"))
```

```
        for r in records
    ]
    path.write_text("\n".join(lines) + ("\n" if lines else ""), encoding="utf-8")
```

- `_read_jsonl` — 손상된 라인(빈 줄, 파싱 실패)은 **조용히 건너뛴다**. 오염된 한 줄이 전체 읽기를 막지 않도록.
- `_write_jsonl` — 콤팩트 모드(`separators=(",", ":")`) 로 직렬화. JSONL 은 한 줄이 한 레코드이므로 indent 를 쓰면 안 된다.

14.4 저장/로드

```
@classmethod
def _find_chat_in_dayfiles(
    cls, username: str, chat_id: str | None
) -> tuple[Path, int] | None:
    """가장 최근 파일부터 검색해서 chat_id 일치 레코드 위치 반환."""
    if not chat_id:
        return None
    d = cls._spec_dir(username)
    for f in sorted(d.glob("*.jsonl"), reverse=True):
        if not cls._SPEC_DAYFILE_RE.match(f.name):
            continue
        recs = cls._read_jsonl(f)
        for i, rec in enumerate(recs):
            if rec.get("chat_id") == chat_id:
                return f, i
    return None

@classmethod
def save_spec(
    cls,
    username: str,
    chat_id: str | None,
    spec: dict[str, Any],
    user_message: str = "",
) -> Path:
    d = cls._spec_dir(username)
    record = cls._ordered_record({
        "saved_at":     _now_iso(),
        "chat_id":      chat_id,
        "user_message": user_message,
        "spec":         spec,
    })

    # 동일 chat_id 레코드가 있으면 in-place 갱신, 없으면 오늘 파일에 append
    found = cls._find_chat_in_dayfiles(username, chat_id)
    if found is not None:
        path, idx = found
        recs = cls._read_jsonl(path)
        recs[idx] = record
    else:
```

```
        path = cls._spec_dayfile(username)
        recs = cls._read_jsonl(path)
        recs.append(record)
    cls._write_jsonl(path, recs)

    # 사람이 한눈에 보기 좋은 latest.json (pretty-printed)
    latest = d / "latest.json"
    latest.write_text(
        json.dumps(record, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    return path

@classmethod
def load_spec(
    cls, username: str, chat_id: str | None = None
) -> dict[str, Any] | None:
    found = cls._find_chat_in_dayfiles(username, chat_id)
    if found is not None:
        path, idx = found
        recs = cls._read_jsonl(path)
        if 0 <= idx < len(recs):
            spec = recs[idx].get("spec")
            if isinstance(spec, dict):
                return spec
    # 폴백: latest.json
    latest = cls._spec_dir(username) / "latest.json"
    if latest.exists():
        try:
            rec = json.loads(latest.read_text(encoding="utf-8"))
        except (json.JSONDecodeError, OSError):
            return None
        if isinstance(rec, dict) and isinstance(rec.get("spec"), dict):
            return rec["spec"]
    return None
```

기능

저장 구조:

```
specs/
└── <sanitized_username>/
    ├── 20260414.jsonl   ← 하루치 명세 로그 (한 줄 = 한 레코드)
    ├── 20260415.jsonl
    └── latest.json      ← 가장 최근 레코드 pretty-print
```

- `save_spec` — `chat_id` 가 이미 어느 dayfile 에 있으면 그 자리 **in-place 덮어쓰기**, 없으면 오늘 dayfile 에 append. 같은 채팅에서의 수정이 새 레코드로 쌓이지 않는다.
- `latest.json` — 모든 저장 시 pretty-print 로 덮어써져서 디버깅/리뷰용 단일 스냅샷 역할.
- `load_spec` — chat_id 로 먼저 찾고, 실패 시 `latest.json` 으로 폴백.

- `_find_chat_in_dayfiles` 는 최신 파일부터 역순 탐색 → 평균 탐색 비용 최소화.

---

15. 응답 포맷팅 유틸

15.1 JSON 파싱 `_parse_json`

```
@staticmethod
def _parse_json(text: str) -> dict[str, Any]:
    stripped = text.strip()
    if stripped.startswith("```"):
        match = re.search(r"```(?:json)?\s*(.*?)```", stripped, re.DOTALL |
re.IGNORECASE)
        if match:
            stripped = match.group(1).strip()

    try:
        parsed = json.loads(stripped)
    except json.JSONDecodeError as exc:
        raise ValueError(str(exc)) from exc

    if not isinstance(parsed, dict):
        raise ValueError("Top-level JSON must be an object")
    return parsed
```

- 프롬프트에서 "순수 JSON 만 출력" 이라고 해도 모델은 종종 ```` ```json ... ``` ```` 펜스로 감싸서 출력한다. 이 함수는 첫 코드 펜스를 벗긴 뒤 파싱.
- top-level 이 object 가 아닌 경우(배열/문자열/숫자)도 `ValueError` 로 통일.

15.2 코드 블록 재포맷 `_format_code_for_webui`

```
@staticmethod
def _format_code_for_webui(text: str) -> str:
    """Open WebUI 호환 코드 블록 변환.

    변환 전:
        ```index.html
        <html>...</html>
        ```
    변환 후:
        📄 `index.html`
        ```html
        <html>...</html>
        ```
    """
    ext_to_lang: dict[str, str] = {
        "html": "html", "htm": "html",
        "css": "css",
        "js": "javascript", "mjs": "javascript", "cjs": "javascript",
        "jsx": "javascript",
```

```
        "ts": "typescript", "tsx": "typescript",
        "py": "python",
        "json": "json",
        "md": "markdown",
        "sh": "bash", "bash": "bash",
        "yml": "yaml", "yaml": "yaml",
        "sql": "sql",
        "rs": "rust",  "go": "go", "java": "java",
        "php": "php",  "rb": "ruby",
        "swift": "swift", "kt": "kotlin",
        "xml": "xml", "svg": "xml", "toml": "toml",
    }

    def _replace(match: re.Match) -> str:
        filepath = match.group(1).strip()
        code = match.group(2)
        ext = filepath.rsplit(".", 1)[-1].lower() if "." in filepath else ""
        lang = ext_to_lang.get(ext, ext)
        return f"\n**📄 `{filepath}`**\n```{lang}\n{code}```"

    # 파일 경로처럼 보이는 코드블록만 변환 (점 포함 & 공백 없음)
    return re.sub(
        r"```([^\n`\s]+\.[^\n`\s]+)\n(.*?)```",
        _replace,
        text,
        flags=re.DOTALL,
    )
```

- 코더 모델이 출력하는 ```파일경로 형식은 Open WebUI 가 언어 힌트로 인식하지 못해 코드가 깨진다.
- 정규식으로 **"점이 포함되고 공백 없는"** 첫 토큰만 파일 경로로 간주해서, 파일명 헤더 + 표준 언어 힌트 (```html 등) 형식으로 치환.
- 확장자 → 언어 매핑은 화이트리스트이고, 매핑에 없으면 확장자 문자열을 그대로 언어 힌트로 사용.

15.3 에러 메시지 변환 `_format_model_error`

```
def _format_model_error(self, stage: str, exc: Exception) -> str:
    message = str(exc)
    lowered = message.lower()
    if (
        "all connection attempts failed" in lowered
        or "connection refused" in lowered
        or "name or service not known" in lowered
        or "nodename nor servname" in lowered
        or isinstance(exc, httpx.ConnectError)
    ):
        return (
            f"{stage} 중 Ollama 서버({self.valves.ollama_url})에 연결하지 못했습니
다.\n"
            "1) Ollama가 실행 중인지 확인하세요 (`ollama serve`).\n"
            "2) `pipelines/config.json`의 `ollama_url`이 올바른지 확인하세요.\n"
            "3) 네트워크/방화벽이 해당 포트(보통 11434)를 막고 있지 않은지 확인하세요."
```

```
        )
    if "404" in message and "model" in lowered:
        return (
            f"{stage} 중 모델을 찾을 수 없습니다: {message}\n"
            "`ollama list`로 설치된 모델 이름을 확인한 뒤 `pipelines/config.json`의
모델명을 수정하세요."
        )
    if "503" in message or "Service Unavailable" in message:
        return (
            f"{stage} 중 모델이 일시적으로 응답하지 않습니다. "
            "잠시 후 다시 시도하거나 `pipelines/config.json`에서 다른 모델로 바꿔 주세
요."
        )
    if "429" in message or "Too Many Requests" in message or "rate_limit" in
lowered:
        return (
            f"{stage} 중 API 사용량 제한에 걸렸습니다. "
            "잠시 후 다시 시도하거나 다른 모델을 사용해 주세요."
        )
    return f"{stage} 중 모델 호출에 실패했습니다: {message}"
```

- 원시 예외 메시지를 4종의 패턴(연결 실패 / 404 모델 없음 / 503 / 429)으로 분류해서 **사용자 액션 가이드가 포함된 한국어 문구**로 변환.
- 매칭 안 되는 예외는 일반 문구로 폴백.
- `stage` 인자는 "요청 분석", "코드 생성", "응답 생성" 등 호출부에서 넘긴 단계명. 어느 단계에서 실패했는지 사용자에게 맥락 제공.

15.4 텍스트 청크 스트리밍 `_stream_text`

```
@staticmethod
def _stream_text(text: str, chunk_size: int = 600) -> Generator[str, None, None]:
    for index in range(0, len(text), chunk_size):
        yield text[index : index + chunk_size]
```

- 완성된 문자열을 고정 크기로 쪼개 yield 하는 단순 청킹.
- `pipe_modify` 에서 사용. 실시간 토큰 스트리밍이 필요 없는 응답(=이미 전부 받은 응답)을 제너레이터 인터페이스에 맞추기 위한 어댑터.